Armand Labernadière
Noémie Niang
Kevin Ojoduma-Quere
Elias Moussa
Raphaël LAURENT

Projet de Numérique Durable

Sommaire

Présentation du projet 4
Proposition de valeur 4
MVP 4
Utilisateurs et leurs rôles 4
Architecture et conception 5
Architecture du projet 5
Diagramme de cas d'utilisation 6
Diagramme de classe 7
Diagramme de séquence 7
Choix technologiques 8
Implémentation 8
Authentification 8
CRUD Utilisateurs 9
CRUD Fiches 10
Mode révision 11
Analyse de l'empreinte carbone 11
Tableau des mesures avant optimisation 11
Tableau des mesures après optimisation : 12
Comparaison avec un site similaire 13
Tests et validation 14
Tests fonctionnels 14
Tests de performance 15
Tests de sécurité 15
Organisation de l'équipe et collaboration 17
Discussion et conclusion 17
Défis rencontrés 17
Compromis entre fonctionnalités et sobriété 17
Piste d'amélioration 18
Regard critique 18
Conclusion 18
Annexe 19

Capture Lighthouse 19

Présentation du projet

Proposition de valeur

Comme projet, nous avons créé une plateforme web destinée principalement aux étudiants, mais utilisable pour tout le monde. Notre plateforme permet aux utilisateurs de créer, consulter et réviser des fiches de cours, de manière simple et efficace pour réduire le stress des révisions et gagner du temps.

MVP

La fonctionnalité principale du site est donc la création et la consultation de fiches de cours. Nous avons alors plusieurs fonctionnalités indispensables :

- Création et modification d'un compte utilisateur
- Création et modification d'une fiche
- Consultation d'une fiche

Nous avons aussi pensé à plusieurs fonctionnalités secondaires comme :

- La suppression d'un compte utilisateur ou d'une fiche
- Un mode spécial de consultation de fiche permettant de masquer le contenu et de le démasquer progressivement pour tester ses connaissances
- Le partage de fiches entre utilisateurs
- Le filtrage de fiches en fonction de leur catégorie

Utilisateurs et leurs rôles

- **Case verte :** Peut
- **Case rouge :** Ne peut pas

	Visiteur	**Utilisateur**	**Administrateur**
Consulter les pages publiques			
Créer un compte			
Se connecter			
Modifier son profil			
Supprimer son compte			
Créer une fiche			

Consulter une fiche			
Modifier une fiche			
Supprimer une fiche			
Modifier un utilisateur			
Supprimer un utilisateur			

Contraintes Green IT

Dès le début de ce projet, nous avons fait certains choix pour respecter des contraintes Green IT :

- Ne pas utiliser de Framework CSS ou JavaScript
- Ne pas utiliser de bibliothèque JavaScript en front-end
- Utiliser une base de données minimale, en utilisant le moins d'attribut descriptif possible
- Utiliser une base de données SQLite plutôt que MySQL
- Ne pas utiliser d'images ou de css trop lourd

Architecture et conception

Architecture du projet

Notre projet est structuré de la façon suivante :

```
│
├── backend/                    # Serveur Node.js + Express
│   ├── middleware/
│   │   └── auth.js             # Middlewares
│   ├── models/
│   │   ├── ficheModel.js       # Requêtes SQL pour les fiches
│   │   └── userModel.js        # Requêtes SQL pour les utilisateurs
│   ├── routes/
│   │   ├── auth.js             # Routes login, logout,
│   │   ├── fiches.js           # Routes CRUD fiches
│   │   └── users.js            # Routes CRUD utilisateurs
│   ├── database.js             # Connexion et initialisation BDD
│   ├── index.js                # Point d'entrée du serveur
```

```
│    └── package.json
│
├── frontend
│    ├── css/                    # Feuilles de style
│    ├── html/                   # Pages HTML
│    ├── javascript /            # Scripts front-end
│
└── docs/                        # Rapport PDF et diagrammes
```

Diagramme de cas d'utilisation

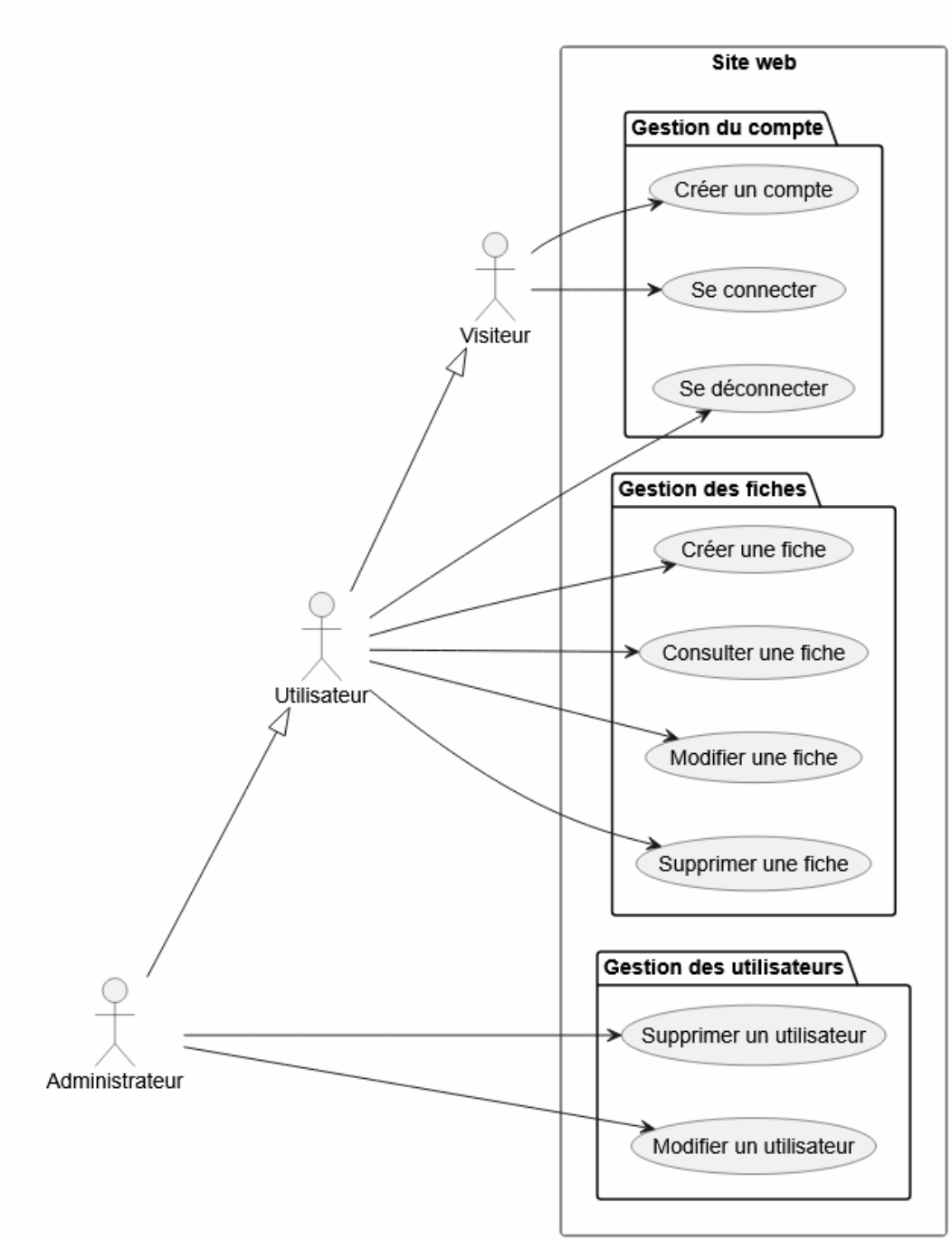


Diagramme des cas d'utilisation

Diagramme de classe

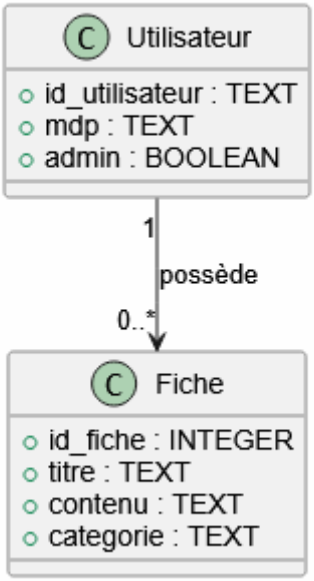


Diagramme de classe

Diagramme de séquence

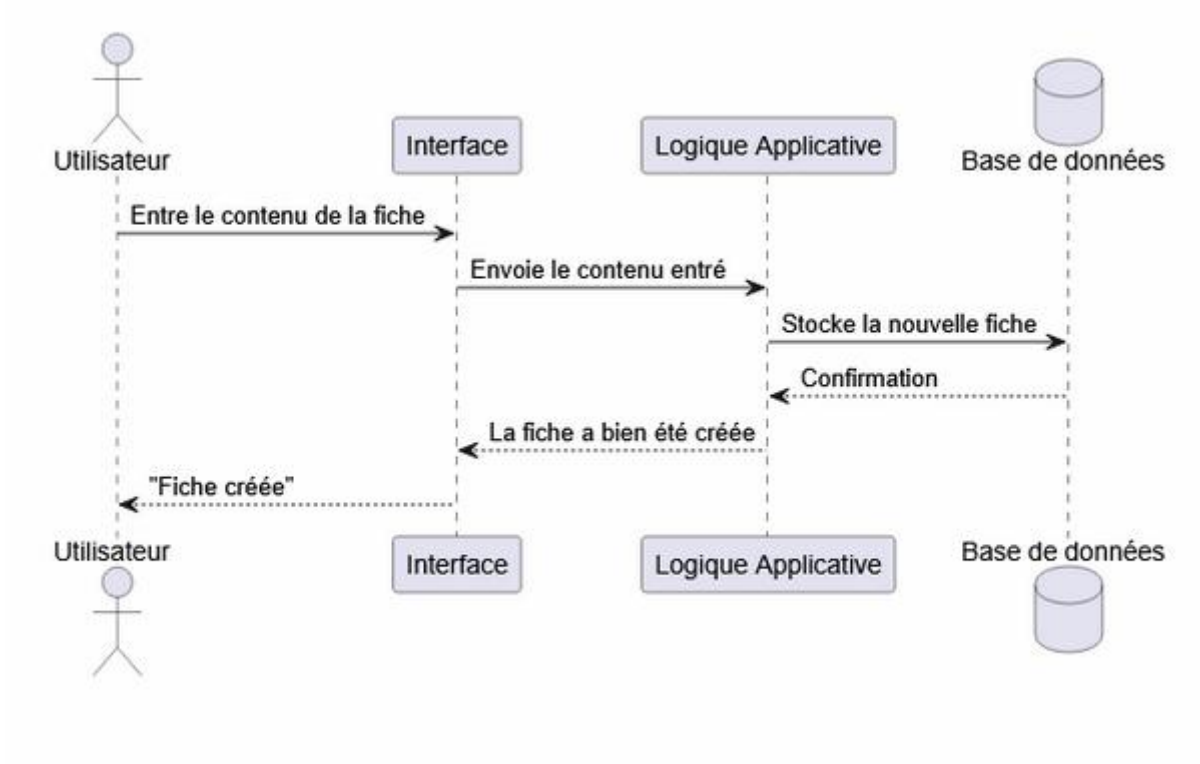


Diagramme de séquence

Choix technologiques

Technologie	Rôle	Justification
HTML, CSS, JavaScript	Front-end	Technologies natives du navigateur, aucun framework lourd n'est obligatoire, pas de dépendance externe. Elles offrent un poids minimal des pages.
Node.js et Express	Back-end	Le runtime donné est léger et performant. De plus, Express nous permet de ne pas surcharger le back-end.
SQLite	Base de données	Adapté pour notre projet qui est de petite taille. Ne demande pas de serveur dédié. Est directement dans le projet.

Implémentation

Notre site possède deux entités métiers : les utilisateurs et les fiches. Notre projet possède donc un CRUD pour chaque, ainsi qu'un système d'authentification et un mode révision pour la consultation de nos fiches.

Notre backend est séparé de la façon suivante :

- Les modèles ficheModel.js et userModel.js qui gèrent la base de données sont dans le dossier models
- Les routes fiches.js, users.js, auth.js qui gèrent la logique applicative dans le dossier routes
- Le fichier auth.js qui contrôle les droits d'accès se trouve dans le dossier middleware
- Le fichier database.js gère la connexion et l'initialisation de la BDD
- Le fichier index.js correspond au point d'entrée du serveur

Authentification

Notre authentification est gérée par deux fichiers : authGuard.js dans le frontend et auth.js dans le backend.

AuthGuard.js est chargé sur toutes les pages protégées (donc toutes nos pages sauf login.html et signup.html) et vérifie lors du chargement de la page si l'utilisateur est connecté. S'il ne l'est pas, l'utilisateur est redirigé vers la page de connexion

Auth.js, lui, possède deux middlewares :

- requireAuth qui vérifie si un utilisateur est connecté
- requireAdmin qui vérifie si un admin est connecté

Ces middlewares sont alors appliqués sur les routes sensibles appropriées pour empêcher tout accès non autorisé.

CRUD Utilisateurs

Le CRUD utilisateurs est réparti entre userModel.js qui gère les requêtes SQL et users.js qui gère les routes.

- **Création d'un compte (avec mot de passe hashé)**

```
router.post("/register", async (req, res) => {
    const { identifiant, mdp, admin } = req.body;
    if (!identifiant || !mdp) {
        return res.status(400).json({ message: "Identifiant et mot de passe
obligatoires." }); // si les champs sont vides
    }
    try {
        // hashage du mot de passe avant stockage
        const mdpHash = await bcrypt.hash(mdp, 10);
        await creerUtilisateur(identifiant, mdpHash, admin);
```

Avec userModel.js qui utilise la requête SQL suivante :

```
        const sql = `INSERT INTO utilisateurs (identifiant, mdp, admin) VALUES
(?, ?, ?)`;
```

- **Requête sql pour lister les utilisateurs (pour admin) :**

```
        const sql = `SELECT identifiant, admin FROM utilisateurs`;
```

- **Modification d'un mot de passe :**

```
    //vérification des droits
    if (utilisateurConnecte.identifiant !== identifiant
&& !utilisateurConnecte.admin) {
        return res.status(403).json({ message: "Accès refusé." });
    }
    if (!mdp) { // le champs doit être remplis
        return res.status(400).json({ message: "Mot de passe obligatoire." });
    }
    try {
```

```
        // Le nouveau mot de passe est hashé
        const mdpHash = await bcrypt.hash(mdp, 10);
        const changes = await modifierUtilisateur(identifiant, mdpHash);
```

Avec userModel.js qui utilise la requête SQL suivante :

```
        const sql = `UPDATE utilisateurs SET mdp = ? WHERE identifiant = ?`;
```

- **Script SQL pour la suppression d'un compte utilisateur :**

```
        const sql = `DELETE FROM utilisateurs WHERE identifiant = ?`;
```

CRUD Fiches

De la même façon que le CRUD utilisateurs, le CRUD fiches est réparti entre ficheModel.js pour les requêtes SQL et fiches.js pour les routes.

- **Création de fiches**

```
router.post("/create", async (req, res) => {
    // récupération de l'identifiant de l'utilisateur
    const id_utilisateur = req.session?.utilisateur?.identifiant;
    // Si pas de session, la requête est refusée
    if (!id_utilisateur) return res.status(401).json({ message: "Non
connecté." });
    const { titre, contenu, categorie } = req.body;

    if (!titre || !contenu) { // les champs ne doivent pas être vides
        return res.status(400).send("Champs manquants");
    }

    try {
        await creerFiche(id_utilisateur, titre, contenu, categorie);
        res.redirect("/fiches.html");
    } catch (err) {
        console.error(err);
        res.status(500).send("Erreur BDD");
    }
});
```

Script SQL :

```
        const sql = `INSERT INTO fiches (id_utilisateur, titre, contenu,
categorie)
                     VALUES (?, ?, ?, ?)`;
```

- **Script SQL pour la consultation des fiches :**

Les fiches sont filtrées par utilisateurs :

```
        const sql = `SELECT id_fiche, titre, categorie FROM fiches WHERE
id_utilisateur = ?`;
```

- **Suppression et modification des fiches**

De la même façon que pour les utilisateurs, on vérifie que la fiche appartient bien à l'utilisateur connecté, et on envoie une erreur si ce n'est pas le cas.

Mode révision

Le mode révision est une fonctionnalité de notre site permettant aux utilisateurs de cacher le contenu de leurs fiches, et leur permet de les révéler phrase par phrase pour pouvoir se tester. Elle est gérée entièrement dans le frontend par le fichier fiche_display.js.

Analyse de l'empreinte carbone

Tableau des mesures avant optimisation

Afin de pouvoir mesurer la performance de notre site, nous avons utilisé plusieurs outils : l'extension lighthouse sur chrome et du site ecoindex.fr.

Nous avons pu relever les résultats suivants :

Pages	**Login.html**	**Signup.html**	**Fiches.html**	**Fiche_display.html**	**Admin.html**
Poids total de la page	3 ko	3 ko	3 ko	3 ko	9 ko
Nombre de requêtes http	3	3	7	6	8
Score EcoIndex	A	A	A	A	A
Note EcoIndex	96	96	95	95	95
CO2/ visite (kgCO2e)	1.08	1.08	1.1	1.08	1.1

Score Lighthouse Performance	100	100	100	100	100
FCP	0.3 s	0.3 s	0.3 s	0.4 s	0.3 s
LCP	0.4 s	0.3 s	0.4 s	0.4 s	0.4 s
TBT	0 ms	0 ms	0 ms	0 ms	0 ms
CLS	0	0	0.012	0.05	0.03

On remarque alors de très bons résultats dans la version initiale de notre projet, notamment grâce à une bonne réflexion d'éco-conception en pré-conception du site web (pas de framework, SQL minimal, ...). Nos scores EcoIndex sont tous en A, et notre score Lighthouse Performance à 100/100.

Nous pouvons tout de même remarquer plusieurs faiblesses et possibles améliorations :

- Même si notre code HTML est bien structuré, notre score SEO n'est pas parfait (90), nous pouvons l'améliorer en ajoutant des balises sémantiques manquantes. Dans notre cas, <footer> ainsi que <meta name='description'>.
- Nous pouvons aussi utiliser cssnano pour minimiser notre code css, et de même avec Terser pour notre code JavaScript.
- Enfin, nous pouvons optimiser notre base de données en ajoutant un index sur id_utilisateur dans la table fiches, pour faciliter les recherches des fiches des utilisateurs.

.

Tableau des mesures après optimisation :

Pages	Login.html	Signup.html	Fiches.html	Fiche_display.html	Admin.html
Poids total de la page	2 ko	2 ko	3 ko	3 ko	6 ko
Nombre de requêtes http	3	3	5	4	8
Score EcoIndex	A	A	A	A	A
Note EcoIndex	96	96	96	96	96

CO2/ visite (kgCO2e)	1.08	1.08	1.08	1.08	1.08
Score Lighthouse Performance	100	100	100	100	100
FCP	0.3 s	0.2 s	0.2 s	0.3 s	0.2 s
LCP	0.3 s	0.3 s	0.3 s	0.3 s	0.3 s
TBT	0 ms	0 ms	10 ms	0 ms	0 ms
CLS	0	0	0.012	0.05	0.017

On observe alors que les optimisations mise en place ont permis de réduire légèrement le poids de nos fichiers et le nombre des requêtes HTTP. Nos scores restent également très bon, avec des notes EcoIndex A, et des scores Lighthouse performance de 100/100. De plus, l'ajout des balises sémantiques manquantes nous a aussi permis d'atteindre un score de 100 SEO Lighthouse sur toutes nos pages.

Comparaison avec un site similaire

Nous avons choisi de comparer nos résultats à ceux d'un site qui offre des fonctionnalités similaires aux notres : partielo.com.

Voici les mesures pour sa page d'accueil :

Pages	**partielo.com**
Poids total de la page	2.028 Mo
Nombre de requêtes http	79
Score EcoIndex	D
Note EcoIndex	43
CO2/ visite (kgCO2e)	2.14
Score Lighthouse Performance	81
FCP	1.3 s
LCP	2.1 s

TBT	110 ms
CLS	0.078

Nous pouvons donc voir que leur page d'accueil est environ 1000 fois plus lourd que nos fichiers, leur nombre de requêtes HTTP est de 79 contres 8 maximum pour nos pages. Ce sont des différences qui peuvent s'expliquer par le manque de sobriété de partielo.com, qui, selon Lighthouse, possède de nombreux problèmes : des fonctionnalités JavaScript obsolètes, l'utilisation de plusieurs polices et des images non optimisées.

C'est une claire différence avec notre site web qui est optimisé pour éviter ces erreurs.

Tests et validation

Tests fonctionnels

Pour valider le bon fonctionnement de notre site, nous avons testé les scénarios suivants sur notre site déployé :

Scénario	Résultat attendu	Statut
Créer un compte utilisateur	Le compte est enregistré dans la BDD, l'utilisateur est redirigé vers l'accueil.	Fait
Créer un compte avec un identifiant existant	L'utilisateur ne peut pas créer de compte, un message d'erreur apparaît.	Fait
Se connecter avec un mauvais mot de passe	L'utilisateur ne peut pas se connecter, un message d'erreur apparaît	Fait
Se connecter avec un compte existant	L'utilisateur est redirigé vers l'accueil	Fait
Supprimer son compte utilisateur	Le compte est supprimé de la BDD, l'utilisateur est redirigé vers la page login.	Fait
Modifier le mot de passe de son compte	La base de données est mise à jour avec le nouveau mot de passe	Fait
Créer une fiche	Une fiche est enregistrée dans la BDD.	Fait
Affichage les fiches d'un utilisateur	On prélève les fiches de la BDD qui appartiennent à	Fait

	l'utilisateur et on les affiche.	
Supprimer une fiche	La fiche est supprimée de la BDD, elle n'apparaît plus dans la liste	Fait
Consultation d'une fiche	L'utilisateur peut consulter sa fiche et activer le mode révision.	Fait
Supprimer un compte utilisateur en tant qu'admin	Le compte est supprimé de la BDD	Fait
Nommer, en tant qu'admin, un utilisateur admin	La base de données est mise à jour, l'autre compte est maintenant admin	Fait
Accès page protégée sans connexion	L'utilisateur est redirigé vers la page de connexion	Fait

Tests de performance

Nous avons repéré comme les trois pages principales de notre site web les pages : login.html, fiches.html et fiche_display.html.

Nous avons alors fait passer ces trois pages sous différents tests : Lighthouse, PageSpeedInsight, réseau 3G et WebPageTest waterfall.

Nous pouvons premièrement voir grâce aux résultats de Lighthouse (voir annexe) que nos pages obtiennent un score de performance de 100/100 avec de bonnes métriques. Notre score de performance PageSpeed Insights est, lui aussi, de 100/100 sur toutes les pages.

En réseau 3G simulé, nous avons observé un chargement de pages et des fonctionnalités plus lentes, tout en restant complètement fonctionnel et sans erreurs.

Enfin, l'analyse de WebPageTest confirme les bonnes performances de notre site, en relevant tout de même une ressource bloquante : notre fichier css.

Tests de sécurité

Nous avons effectué plusieurs vérifications de sécurités dont :

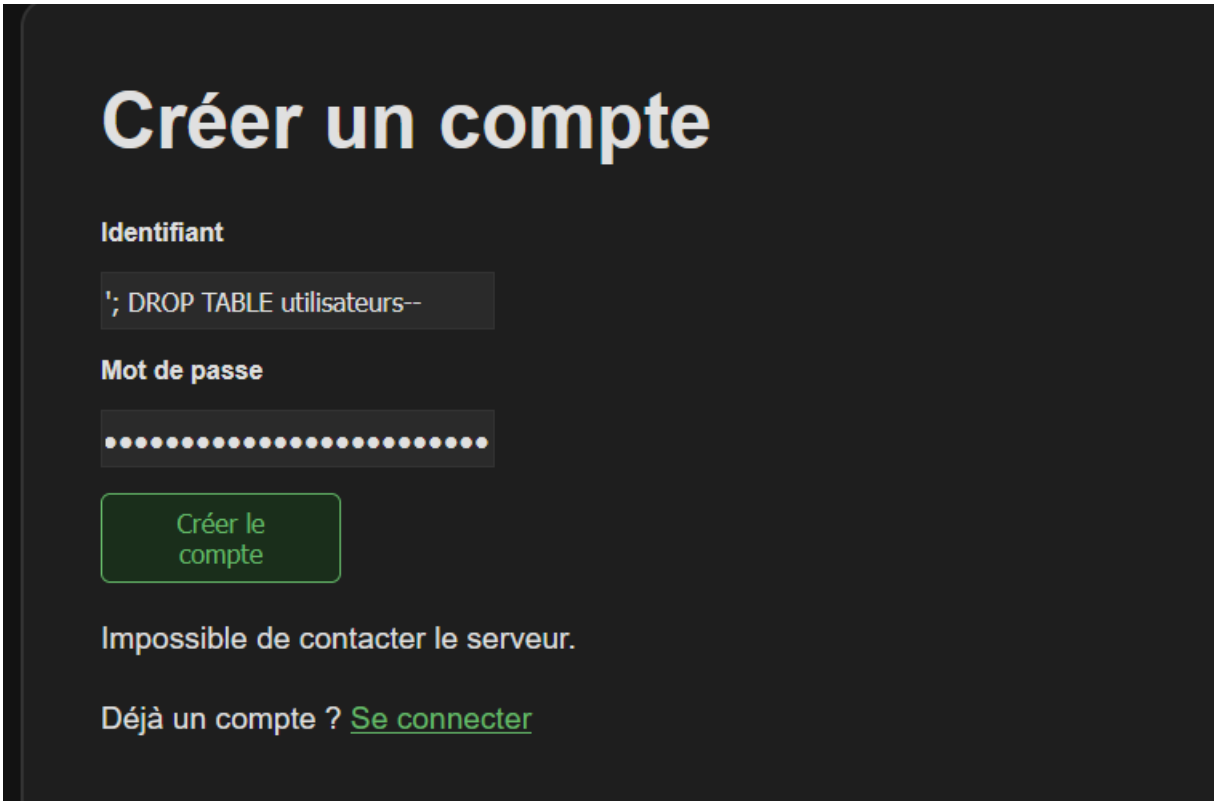
Mot de passe hashés en base de données

Comme vu dans l'image ci-dessous, les mots de passes stockés dans la table utilisateur sont sous forme hashée.

	identi...	mdp	admin
1	admin	$2b$10$IZd8lbV0ZkbfuK2n1ITTvOUTopzZJiPXc.vC9gy66...	✓
2	a	$2b$10$gSvIrli2QPgcV1ibF7EpL.cgLYsUmsNBa15P4OzG...	
3			

Test injection SQL

Nous avons testé une injection SQL classique en entrant '; DROP TABLE utilisateurs— dans les deux champs du formulaire de connexion. Comme on peut le voir ci-dessous, la requête n'est pas passée, et un message d'erreur s'affiche.



Pas de variables sensibles sur GitHub

Aucune variable sensible n'est présente dans le code. Les valeurs d'exemple sont documentées dans .env.example.

Pages protégées sans connexion

Grâce à authGuard.js, quand on essaye d'accéder à une page protégée sans être connectée, on est bien redirigé vers la page login.html (les pages non protégées étant login et signup, les pages protégées sont toutes les autres).

Organisation de l'équipe et collaboration

Lors de ce projet, nous avons organisé le travail de la façon suivante :

- Armand Labernardière : diagrammes UML, cas d'utilisation, mesures écologiques et Optimisation Green IT

- Raphaël Laurent : back-end fiches, admin et Optimisation Green IT

- Noémie Niang : back-end utilisateurs, authentification (login/logout), middlewares et admin

- Kevin Ojoduma-Quere : diagrammes UML, wireframes, mesures écologiques, correction de code et déploiements

- Elias Moussa : CSS, design de l'interface, wireframes et déploiement

Nous avons utilisé GitHub lors de ce projet, nous permettant de sectionner notre travail en une branche par fonctionnalité.

Discussion et conclusion

Défis rencontrés

Le défi principal que nous avons rencontré lors de ce projet fut la gestion de l'authentification et des middlewares : la mise en place des sessions, la protection des routes et la bonne synchronisation du frontend et du backend ont pris du temps avant de fonctionner correctement.

Nous avons aussi eu des difficultés avec le déploiement sur Render qui n'était pas complétement compatible avec SQLite. Nous avons dû provoquer la recompilation des modules en ajoutant --build-from-source à la commande de build. Nous avons aussi dû changer la manière dont on gère une base de données, en implémentant l'initialisation automatique de celle-ci.

Compromis entre fonctionnalités et sobriété

Ce projet nous a permis de réaliser comment créer un site web dans une démarche d'éco-conception pouvait être contraignant, mais pas impossible.

Nous avons dès le début compris qu'il fallait éviter les images et le css excessif, tout en réussissant à rendre notre site web plaisant esthétiquement. Ça a été notre principal compromis : adopter une apparence sobre, pour pouvoir optimiser au mieux notre site.

Piste d'amélioration

Même si nous trouvons notre projet plutôt bien réussi, nous avons tout de même repéré plusieurs améliorations.

Notamment, la migration vers une base de données comme PostgreSQL ou MySQL pour avoir des données qui persistent entre les déploiements de l'application.

De plus, nous n'avons pas eu le temps d'implémenter plusieurs fonctionnalités secondaires prévues dans le MVP, soit la possibilité de partager des fiches entre utilisateurs, et la possibilité de filtrer ses fiches en fonction de leur catégorie.

Regard critique

Maintenant que nous avons fini ce projet, nous sommes assez satisfaits de nos choix d'éco-conception pris lors du début de ce projet : ils nous ont permis d'atteindre un très bon score très rapidement.

Nous pensons tout de même que nous aurions pu mieux réfléchir à la structure de notre projet, ce qui nous aurait permis de pouvoir développer la partie backend plus tranquillement.

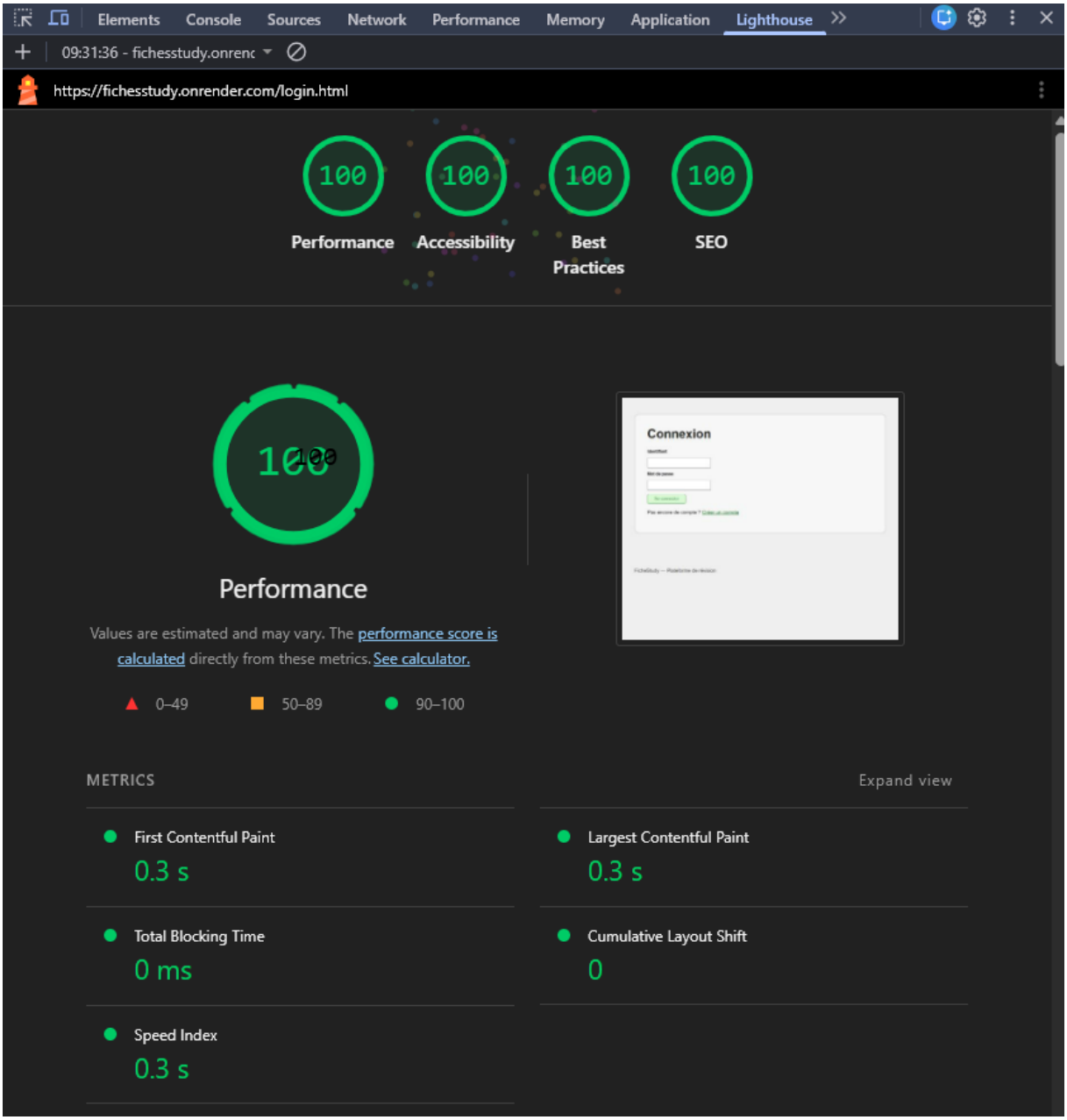
Conclusion

Pour conclure, ce projet nous aura permis de prendre conscience de l'accessibilité de l'éco-conception dans le développement web. C'est quelque chose qui prend peut-être un peu de temps en plus, mais qui donne de bons résultats pour le climat, mais aussi pour la performance du site web en lui-même.

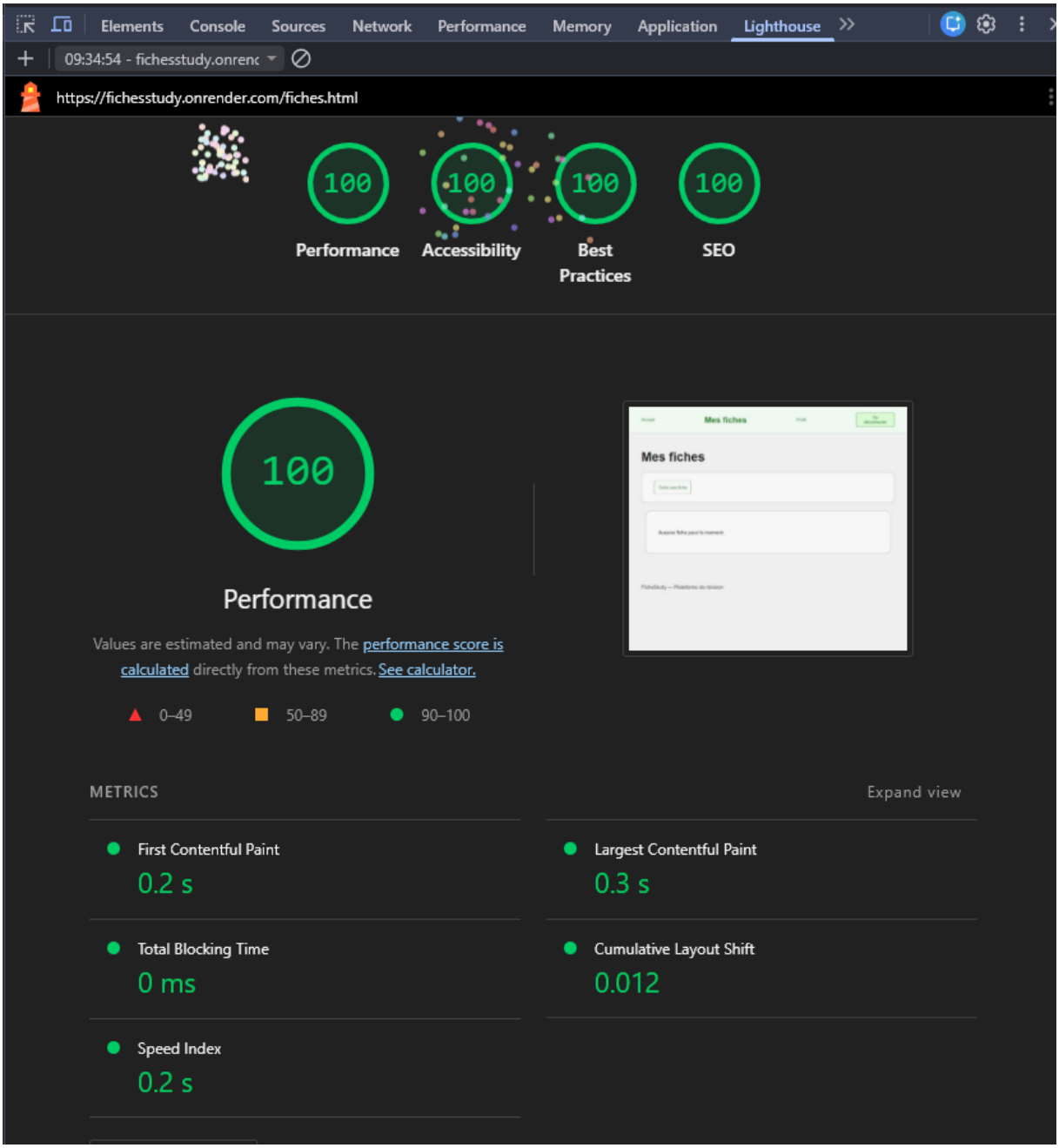
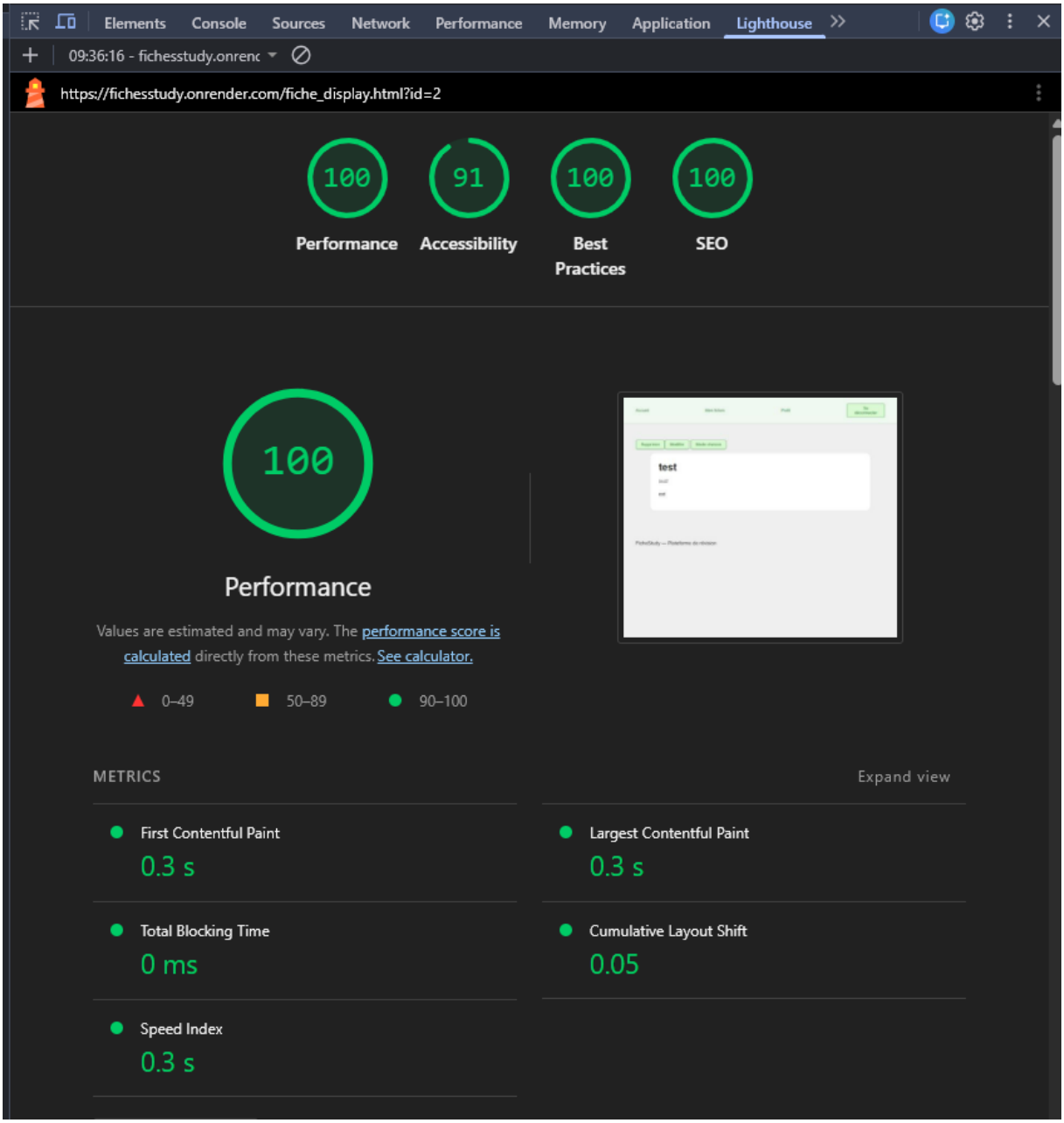
Annexe

Capture Lighthouse

Login.html :



Fiches.html :


Elements
Console
Sources
Network
Performance
Memory
Application
Lighthouse
09:34:54 - fichesstudy.onrenc
https://fichesstudy.onrender.com/fiches.html
100
Performance
100
Accessibility
100
Best Practices
100
SEO
100
Performance
Values are estimated and may vary. The performance score is calculated directly from these metrics. See calculator.
0–49
50–89
90–100
Mes fiches
Mes fiches
METRICS
Expand view
First Contentful Paint
0.2 s
Largest Contentful Paint
0.3 s
Total Blocking Time
0 ms
Cumulative Layout Shift
0.012
Speed Index
0.2 s

Fiche_display.html :